Objectives

- To perform reading data from standard input and writing data to standard output with their respective file descriptors using read() and write() system calls.
- To perform open and close operations on a file using open() and close() system calls.
- To demonstrate an understanding of open flags
- To perform reading from a file and writing data to the file with their respective file descriptors using read() and write() system calls.

Pre-Lab Theory

1. ***read() synopsis:***

*int read(int fd, char*buffer, int size)*

Description:

read from file descriptor fd

file descriptor:

Represents a handle for the opened file. It is the index of the process file table entry.

Default Descriptors in the process file table with symbolic names:

Standard output: *STDOUT_FILENO*

Standard input: *STDIN_FILENO*

Standard Error: *STDERR_FILENO*

buffer: *The data read from the file descriptor representing a file/console is stored in the buffer*

size: *The size/chunk/block to be read in the buffer*

return*: The number of bytes successfully read*

2. ***write() synopsis:***

*int write(int fd, char*buffer, int size)*

file descriptor:

Represents a handle for the opened file. It is the index of the process file table entry.

Default Descriptors in the process file table with symbolic names:

Standard output: *STDOUT_FILENO*

Standard input: *STDIN_FILENO*

Standard Error: *STDERR_FILENO*

buffer: *The data stored in the buffer is written to the file represented by the file descriptor*

size: *The size/chunk/block to be written to the file*

return*: The number of bytes successfully written*

3. ***open() synopsis:***

int open(char path, int mode_flags)*

path: *The location of the file*

mode_flags*: The access flags, O_RDONLY, O_WRONLY, O_RDWR, O_CREAT*

return: *The file descriptor of the file if successfully opened.*

4. ***Close(int fd):***
   int close(int fd)

In-Lab Tasks

header file*: unistd.h, stdio.h*

System call: read(), write(), open(), close()

Task1(a)

Include the header files and the following line of code in your program. Compile and run the code and describe its behavior.

```
char buffer[] = “Welcome”;
write(STDOUT_FILENO, buffer,
sizeof(buffer);
```

Brief the working/output:

Welcome

Task 1(b)

Include the header files, declare the required variables, and the following line of code in your program. Compile and run the code and brief the working of the code.

```
bytes_read = read(STDIN_FILENO,buffer,
sizeof(buffer)); if(bytes_read < 0) printf(“read()
ERROR”);exit(); bytes_write = write(STDOUT_FILENO,
buffer, bytes_read); if(bytes_write < 0)
printf(“write() ERROR”);exit();
```

Brief the Working/output:

```
muhammad-ahmad@latitude-7490:~/lab3$ ./task1b
  hello
  hello
```

Task 2(a)

first, create the file "policy.data" in the present working directory using the open() system call in read/write mode.

Error-free Code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    // O_CREAT -> create file if not exists
    // O_RDWR  -> read/write mode
    // 0644    -> permission (rw-r--r--)
 fd = open("policy.data", O_CREAT | O_RDWR, 0644);

    if (fd < 0) {
        perror("Error creating/opening file");
        return 1;
    }

printf("File 'policy.data' created successfully with
fd = %d\n", fd);

    close(fd);
    return 0;
}
```

Task 2(b)

Write the statement "Honesty is the Best Policy" in the above-created file using a write system call

Error-free Code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main() {
    int fd;
    char buffer[] = "Honesty is the Best Policy";
```

```
            fd = open("policy.data", O_WRONLY);
            if (fd < 0) {
                perror("Error opening file");
                return 1;
            }
// sizeof(buffer)-1 because of eof character,instead use strlen
            write(fd, buffer, sizeof(buffer)-1);

  printf("Message written successfully into 'policy.data'\n");

            close(fd);
            return 0;
        }
```

Task 2(c) Create another file, "backup.data" in the directory one level above the present working directory with write mode flag.

Error-free Code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;

    // "../backup.data" means one level above
    fd = open("../backup.data", O_CREAT | O_WRONLY, 0644);

    if (fd < 0) {
        perror("Error creating/opening file");
        return 1;
    }

    printf("File '../backup.data' created successfully\n");

    close(fd);
    return 0;
}
```

Task 2(d) Read data from "policy.data" file and write to the "backup.data" file .
Error-free Code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd1, fd2;
    char buffer[100];

    // 1. open policy.data in read-only mode
    fd1 = open("policy.data", O_RDONLY);
    if (fd1 < 0) {
        perror("Error opening policy.data");
        return 1;
    }

    // 2. open backup.data in write-only mode
    fd2 = open("../backup.data", O_WRONLY);
    if (fd2 < 0) {
        perror("Error opening backup.data");
        close(fd1);
        return 1;
    }
    // counting no of bytes for writing
     int n = read(fd1, buffer, sizeof(buffer));
     // write data in range of counted bytes
     write(fd2, buffer, n);

   printf("Data copied from policy.data to ../backup.data\n");

    close(fd1);
    close(fd2);

    return 0;
}
```

Task 3(a): open the file "policy.data" again. Append and repeat the same data in "policy.data" file 10 times in separate lines using write() system call and for loop.

Close both the files.

Error Free Code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd;
    char buffer[] = "Honesty is the Best Policy\n";
    int i;

    // open in append + write mode
    fd = open("policy.data", O_WRONLY | O_APPEND);
    if (fd < 0) {
        perror("Error opening file");
        return 1;
    }

    // write the message 10 times
    for (i = 0; i < 10; i++) {
         n = write(fd, buffer, strlen(buffer));
        if (n < 0) {
            perror("Error writing to file");
            close(fd);
            return 1;
        }
    }

    printf("Data appended 10 times in 'policy.data'\n");

    close(fd);
    return 0;
}
```

Task 3(b):open the "policy.data" file and "backup.data" file in the appropriate mode and copy the whole file "policy.data" to the "backup.data" file.
Detect End-of-File during reading data from source file. Print the number of bytes written.

Error-free Code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main (){
     int fd1 , fd2;
     int  no_bytes = 0;
     char buffer [256];
// opening files in appropropriate mode
     fd1 = open ("policy.data" ,O_RDONLY);
     if (fd1 < 0) {
          perror ("failed to open file policy.data");
     }
     fd2 = open ("backup.data" ,O_WRONLY);
     if (fd1 < 0) {
          perror ("failed to open file backup.data");
     }
// read and write wrt buffer space until stream end
     int n ;
     while (n=read (fd1, buffer, sizeof(buffer)) > 0){
          if ( write (fd2 , buffer , n != n ){
               perror (“write error”);
               close(fd1);
               close(fd2);
          };
           no_bytes += n;
     }
     printf ("no.of bytes written = %d bytes" ,no_bytes);
     close(fd1);
     close(fd2);

     return 0;
}
```